

Pydantic — Validation at the Boundary

How the node turns untrusted external data into typed, validated objects at run time — using the same type-hint syntax Chapter 5 only checked statically.

SUBJECT hathor-core · Part I · Track C (the stack)

CHAPTER 18 · Foundations · The Stack

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Pydantic v2 · Models · Runtime validation · Type coercion · Validators · ConfigDict · frozen / extra=forbid · Parse don't validate

Table of Contents

Chapter 18 — Pydantic: Validation at the Boundary	3
18.1 The problem: the inside is typed, the outside isn't	3
18.2 The complement to mypy: same syntax, opposite timing	4
18.3 The Pydantic model	4
18.4 Configuring models: encoding invariants	5
18.5 How hathor-core uses it, and why Pydantic	6
18.6 Bridge — Pydantic across the project	7
Recap	7

Chapter 18 — Pydantic: Validation at the Boundary

What you'll learn in this chapter

- The boundary problem: external data (config, CLI args, API requests) arrives untyped and untrusted, and must be checked before the node acts on it.
- The complementary roles of **mypy** (Chapter 5, static, before run) and **Pydantic** (run time, on real data) — same type-hint syntax, opposite timing.
- The core model of **Pydantic**: a **model** is a class whose fields are type-annotated, and constructing it **validates and coerces** the input or raises.
- Model configuration that encodes invariants — **extra='forbid'**, **frozen=True** — and custom **validators**.
- How `hathor-core` uses it (`RunNodeArgs`, `HathorSettings`, a custom `BaseModel`), and **why Pydantic** over hand-written checks.

Chapter 5 taught type hints and `mypy` — a static safety net that checks the code's internal consistency before it runs. But `mypy` cannot help with data the node receives while running: a settings file someone edited, command-line flags, a JSON body from an API client. That data is untyped text and entirely untrusted, and a wrong value can corrupt the node deep in its operation. **Pydantic**¹ is the tool that guards that boundary: it validates external data at run time, turning it into typed, trustworthy objects — and it does so using the same type-hint syntax `mypy` reads. This chapter completes the typing story Chapter 5 began.

It follows the §5 primer shape; the trade-off discussion closes it.

18.1 The problem: the inside is typed, the outside isn't

Inside the node, Chapter 5's hints and `mypy` keep types consistent. But every program has a **boundary**² where data crosses in from the outside world, and at that boundary all guarantees vanish:

- A **settings** YAML file is just text until parsed — a number might be a string, a required field might be missing, a typo'd key might be silently present (Chapter 22).

- **CLI arguments** arrive as strings — `--listen 8080` gives the string `"8080"`, not the integer `8080`.
- An **API request body** is whatever a client sent — possibly malformed, possibly malicious.

Two dangers follow. First, wrong types: code expecting an `int` gets a `str` and fails far from the source (the dynamic-typing pain of Chapter 5 §5.1, now with attacker-influenced data). Second, invalid values: a number out of range, an unknown field, a missing required key. You want to catch all of this **at the boundary, immediately**, with a clear error — not deep in the call stack hours later. This is the **fail-fast**³ principle, and it is what a validation library provides.

18.2 The complement to mypy: same syntax, opposite timing

Here is the relationship that makes Pydantic click, and it directly extends Chapter 5. Recall that type hints are **ignored at run time** (§5.2) — mypy reads them before the program runs, then they do nothing. Pydantic uses the identical hint syntax but gives it run-time teeth:

mypy (Chapter 5)	Pydantic (this chapter)
reads hints BEFORE running	enforces hints WHILE running
checks code's consistency	checks real data's validity
no effect at run time	raises on bad data at run time
guards: the inside	guards: the boundary

They are complementary, not competing. mypy ensures the code is internally consistent before it ships; Pydantic ensures the data crossing in conforms before the code touches it. A field annotated `port: int` means, to mypy, "treat this as int"; to Pydantic, "at run time, make this an int or raise." Same `: int`, two enforcement moments. Once you see this, the two halves of the typing story fit together.

18.3 The Pydantic model

The central concept is the **model**⁴: a class inheriting from Pydantic's `BaseModel`, whose fields are declared with type hints. Constructing the model from raw data triggers validation.

```

from pydantic import BaseModel

class ServerConfig(BaseModel):
    host: str
    port: int
    debug: bool = False          # a default makes the field optional

cfg = ServerConfig(host="localhost", port="8080")  # note: port is a STRING
print(cfg.port, type(cfg.port))                  # 8080 <class 'int'> ← coerced!

ServerConfig(host="x", port="not-a-number")      # raises ValidationError

```

Two behaviors to notice:

- **Coercion.** `port="8080"` (a string) became the integer `8080`. Pydantic coerces compatible types where it safely can — directly solving the "CLI args arrive as strings" problem of §18.1.
- **Validation with clear errors.** `port="not-a-number"` raises a `ValidationError` describing exactly which field failed and why — at construction, the boundary, not later. This is fail-fast made concrete.

This embodies a principle often summarized as "**parse, don't validate**"⁵: rather than scattering `if`-checks throughout the code, you parse the untrusted input once into a typed object at the boundary; thereafter the rest of the program works with a value that is guaranteed well-formed by its type. The validation happens in exactly one place, and downstream code can trust what it holds. (`hathor-core` uses **Pydantic v2**, the current major version, with APIs like `field_validator`, `model_validator`, and `ConfigDict`.)

18.4 Configuring models: encoding invariants

Beyond field types, a Pydantic model can be configured to enforce stronger invariants — and this is where it connects to the design ideas of earlier chapters. `hathor-core` defines a custom base model (`hathor/utils/pydantic.py:79`) that sets two config options:

```

class BaseModel(pydantic.BaseModel):
    model_config = ConfigDict(extra='forbid', frozen=True)  # utils/pydantic.py:83

```

- `extra='forbid'` — reject any input field the model does not declare. A typo'd or unexpected key raises instead of being silently ignored. This is the safety property you met in Chapter 22 for settings: a malformed profile fails at load, not later.
- `frozen=True` — make instances **immutable** after construction (assigning to a field raises). This is the read-only invariant of Chapter 1 (§1.4) and Chapter

22, enforced by the library rather than by convention: once parsed, the object cannot be accidentally mutated.

(Note a deliberate distinction the codebase makes: this custom `BaseModel` is `frozen=True`, but `HathorSettings` — Chapter 22 — sets only `extra='forbid'`, not `frozen`. The settings object's read-only-ness rests on `extra='forbid'` plus convention, while this general-purpose base is fully frozen. Both are honest about what they guarantee.)

Custom validators go further than types allow. A field validator runs your own logic on a field's value (e.g. "this weight must be positive"); a model validator checks relationships between fields (e.g. Chapter 22's rule that a proof-of-authority network must not also configure a block reward). You met both in the settings chapter; they are how domain rules — not just type correctness — get enforced at the boundary.

18.5 How `hathor-core` uses it, and why Pydantic

Two representative models, both at the boundary:

- **RunNodeArgs** (`hathor_cli/run_node_args.py:24`) — the parsed, validated form of the node's command-line arguments (Chapter 21). Raw CLI strings become a typed object the rest of the boot path can trust.
- **HathorSettings** (`hathor/conf/settings.py:32`) — the validated network configuration loaded from a YAML profile (Chapter 22), with field and model validators enforcing network rules.

The project also wraps Pydantic with helpers — a custom `BaseModel` (§18.4) and a `Hex` generic for hex-encoded byte fields in JSON — so the same validation discipline applies consistently across the codebase.

Why Pydantic, and not the alternatives?

- **vs. hand-written validation.** You could write `if not isinstance(port, int): raise ...` everywhere. It's verbose, error-prone, easy to forget a case, and scatters the rules across the code. Pydantic declares the rules once in the model's types and config, generates the checks, and produces uniform, descriptive errors. Less code, fewer gaps.
- **vs. dataclasses / TypedDict.** Standard `@dataclass` (and `TypedDict`) give you typed structure but do **no run-time validation** — a `dataclass` happily accepts a string for an `int` field, because (Chapter 5) hints are ignored at run time. They organize data; they don't guard the boundary. Pydantic adds the enforcement they lack.
- **vs. other validation libraries (marshmallow, cerberus).** These predate Pydantic's type-hint-driven style and use separate schema definitions.

Pydantic's advantage is that the type hints are the schema — one declaration serves mypy, the reader, and run-time validation — which is exactly the unification this chapter is about.

The honest trade-off: Pydantic adds a dependency and a small run-time cost (validation isn't free), and its coercion rules occasionally surprise (knowing when it coerces vs. rejects takes learning). In return you get boundary safety with almost no boilerplate, expressed in the same type language as the rest of the code. For a node ingesting untrusted config and requests, that safety is essential.

18.6 Bridge — Pydantic across the project

Bridge — Pydantic in the project and the stack:

- **It completes Chapter 5's typing story.** Same hint syntax, enforced at run time instead of statically — the static/dynamic halves of typing — **Chapter 5**.
- **It guards settings.** `HathorSettings` validates the network profile at load, fail-fast — **Chapter 22**.
- **It guards CLI args.** `RunNodeArgs` turns raw flags into a trusted typed object — **Chapter 21**.
- **It enforces invariants from Chapter 1.** `frozen=True` / `extra='forbid'` make read-only and no-unknown-fields library-enforced, not convention — **Chapters 1 & 22**.
- **mypy understands it.** The `pydantic.mypy` plugin (Chapter 20) lets the type checker reason about models — **Chapter 20**.
- **It is declared by Poetry.** `pydantic = "^2.0"` — **Chapter 13**.

Recap

Concept	What it is	Why it matters
boundary	where external data crosses in	where guarantees vanish, must re-check
mypy vs Pydantic	static (before run) vs run-time	same hints, complementary timing
model	<code>BaseModel</code> subclass with typed fields	the unit of validation
coercion	safe type conversion of input	<code>"8080"</code> → <code>8080</code>

Concept	What it is	Why it matters
ValidationError	clear failure at construction	fail-fast at the boundary
<code>extra='forbid'</code>	reject unknown fields	typos fail loudly
<code>frozen=True</code>	immutable after construction	read-only invariant, enforced
validators	custom field/model rules	domain rules, not just types
why Pydantic	hints are the schema	vs hand-checks / dataclasses

Pydantic guards the node's boundary: it turns untrusted external data — settings, CLI arguments, API bodies — into typed, validated, optionally-immutable objects at run time, raising a clear error the moment something is wrong rather than failing deep in the system later. It uses the exact type-hint syntax of Chapter 5 but enforces it while running, making it the run-time complement to mypy's static checks, and its model configuration turns design invariants (read-only, no unknown fields, domain rules) into library-enforced guarantees. It is chosen over hand-written checks and plain dataclasses because the type hints are the schema. The next chapter covers the tool that produces some of that boundary data in the first place — `configargparse`, which unifies command-line flags, environment variables, and config files.

1. Pydantic is a Python library for data validation using type hints. A model class declares typed fields; constructing it validates and coerces input data or raises a descriptive error. `hathor-core` uses Pydantic v2. ↩
2. A boundary is any point where data enters a program from outside (files, network, user input, other systems). Data at the boundary is untyped and untrusted and must be validated before the program relies on it. ↩
3. Fail-fast is the principle of detecting and reporting an error as early and as close to its source as possible — here, validating input at the boundary so a bad value raises immediately with a clear message rather than causing an obscure failure later. ↩
4. A Pydantic model is a class subclassing `BaseModel` whose attributes are declared with type hints. It validates and coerces data on construction and provides serialization to/from dicts and JSON. ↩
5. "Parse, don't validate" is a design maxim: instead of repeatedly checking untrusted data throughout the code, parse it once at the boundary into a type that cannot hold invalid data, so downstream code can trust it by construction. ↩
6. Pydantic v2 is the current major version, with a rewritten core and APIs including `ConfigDict`, `field_validator`, and `model_validator`. It is faster than v1 and is what `hathor-core` uses. ↩